

SOFTWARE ENGINEERING

ASSIGNMENT – 03

Architecture and Design

AdaptIQ — AI-Powered Personalized Study Assistant

Team Members:

Samia Imran – BSCS24154-A

Ali Amir – BSCS24137-A

1. Introduction

AdaptIQ is an AI-driven personalized study assistant designed to help students improve their learning outcomes. The system evaluates quiz performance using machine learning, identifies weak and strong areas, and provides tailored study recommendations. It also supports collaborative learning through note sharing, real-time document editing, and peer resource discovery.

Core Features:

- Quiz attempt and automated evaluation
- ML-based performance analysis and weak-area detection
- Personalized quiz and study material recommendations
- Progress tracking with visual charts
- Notes upload, search, rating and review
- Real-time collaborative document editing

Technology Stack:

Layer	Technology
Frontend	React.js (with Tailwind CSS)
Backend	Python – FastAPI
Database	PostgreSQL (relational data) + Redis (caching/sessions)
ML Engine	Scikit-learn / TensorFlow (quiz analysis & recommendations)
Real-time Collab	WebSocket (via FastAPI / Socket.IO)
Auth	JWT-based authentication
Deployment	Docker + cloud hosting (AWS / GCP)

2. Architectural Design

2a) Chosen Architectural Style

Selected Style: Layered (N-Tier) Architecture

AdaptIQ uses a three-tier layered architecture:

- Presentation Layer: React.js frontend
- Application / Business Logic Layer: FastAPI backend with ML engine
- Data Layer: PostgreSQL + Redis

Justification:

- Separation of concerns: UI, business logic, and data are cleanly decoupled, making the system easier to maintain and test.

- Clear responsibility boundaries: Each layer has a well-defined role, reducing coupling between modules.
- Ease of replacement: Any layer can be swapped independently without affecting others.
- Scalability: Individual layers can be scaled horizontally without redesigning the entire system.
- Team familiarity: The team has experience with React + FastAPI + PostgreSQL, reducing development risk.

2b) Architectural Overview

The system is divided into the following major components. Components communicate via REST API (HTTP/JSON), and WebSocket is used exclusively for real-time collaboration.

Component	Responsibilities	Technology
React Frontend	Renders UI, handles user input, displays recommendations and charts, real-time editor UI.	React.js, Tailwind CSS, Socket.IO client
FastAPI Backend	Handles all REST endpoints, orchestrates business logic, delegates to ML engine, manages WebSocket sessions.	Python, FastAPI, Pydantic
ML Engine	Analyses quiz scores, identifies weak topics, generates personalized study recommendations.	Scikit-learn / TensorFlow
Auth Module	Handles user registration, login, JWT issuance and validation.	FastAPI, JWT, bcrypt
Notes Service	Manages upload, tagging, searching, and rating of shared notes.	FastAPI, PostgreSQL
Collab Service	Manages shared documents, broadcasts real-time edits to connected peers.	WebSocket, Redis pub/sub
PostgreSQL DB	Persistent storage for users, quizzes, results, notes, reviews, documents.	PostgreSQL
Redis	Caches session data, stores collaboration state, pub/sub broker.	Redis

Component Interaction Flow:

The React frontend sends HTTP requests to FastAPI REST endpoints. The backend validates the JWT token, processes the request (calling the ML engine or database as needed), and returns a JSON response. For collaboration, the frontend connects via WebSocket; the backend uses Redis pub/sub to broadcast document changes to all connected clients instantly.

3. Detailed Design

Three core modules are described in detail below.

Module 1: ML Performance Analysis & Recommendation Engine

3a) Module Description

This module is the core intelligence of AdaptIQ. It evaluates a student's quiz results, computes per-topic performance scores, identifies weak areas, and recommends targeted study materials and follow-up quizzes.

- Inputs: Quiz ID, Student ID, list of answers submitted by the student.
- Outputs: Quiz score, per-topic performance breakdown, list of recommended quizzes/materials.
- Dependencies: Quiz Database, User Performance History table, ML Model (trained offline).

3b) Internal Structure — Pseudo-code

```
FUNCTION evaluateQuiz(studentId, quizId, answers):
    correctAnswers = DB.getCorrectAnswers(quizId)
    score = 0
    topicScores = {}
    FOR each (answer, correct) in zip(answers, correctAnswers):
        IF answer == correct THEN score += 1
        topicScores[answer.topic] += getScore(answer, correct)
    DB.saveResult(studentId, quizId, score, topicScores)
    RETURN score, topicScores

FUNCTION analyzePerformance(studentId):
    history = DB.getQuizHistory(studentId)    // last N quizzes
    featureVector = buildFeatureVector(history) // per-topic averages
    weakTopics = []
    FOR topic, avgScore in featureVector:
        IF avgScore < THRESHOLD THEN weakTopics.append(topic)
    RETURN weakTopics

FUNCTION recommend(studentId):
    weakTopics = analyzePerformance(studentId)
    recommendations = MLModel.predict(weakTopics) // trained recommender
    RETURN DB.fetchMaterials(recommendations)
```

Module 2: Notes Sharing Service

3a) Module Description

This module manages the community-driven notes repository. Students can upload study notes, tag them with metadata, search/filter the repository, and rate or review notes shared by others.

- Inputs (Upload): File, title, description, subject/course/topic tags, uploader student ID.
- Inputs (Search): Keywords, subject, course, topic filter values.
- Inputs (Rate): Note ID, student ID, star rating (1–5), text review.
- Outputs: Saved note record, search results list, updated average rating.

- Dependencies: File Storage (S3/local), Notes table, Reviews table, User table.

3b) Internal Structure — Class Diagram (described)

Class	Attributes	Key Methods
Note	id, title, description, filePath, subject, course, topic, uploaderId, averageRating, createdAt	upload(), delete(), updateRating()
NoteSearch	keyword, subject, course, topic	search() → List[Note], applyFilters()
Review	id, noteId, reviewerId, starRating, text, createdAt	submit(), validate()

The NoteSearch class queries the PostgreSQL notes table using full-text search on title/description combined with equality filters on subject, course, and topic. Results are sorted by average rating descending. On each new review submission, Note.updateRating() recalculates the average from all stored reviews.

Module 3: Real-Time Collaboration Service

3a) Module Description

This module enables multiple students to edit a shared document simultaneously with live cursor tracking and change synchronization. It is built on WebSocket connections and a Redis pub/sub channel per document.

- Inputs: User join event (docId, userId), text edit event (docId, delta, userId).
- Outputs: Broadcast updated document state to all collaborators, cursor positions.
- Dependencies: WebSocket server (FastAPI), Redis (pub/sub), Documents table (PostgreSQL for persistence).

3b) Internal Structure — Pseudo-code

```

ON client_connect(socket, docId, userId):
    joinRoom(socket, docId)
    docState = Redis.get('doc:' + docId) OR DB.loadDocument(docId)
    socket.emit('init', docState)
    broadcastToRoom(docId, 'user_joined', userId)

ON client_edit(socket, docId, delta, userId):
    newState = applyDelta(Redis.get('doc:' + docId), delta)
    Redis.set('doc:' + docId, newState)           // fast in-memory update
    Redis.publish('room:' + docId, {delta, userId}) // broadcast via pub/sub
    scheduleDBPersist(docId, newState)           // debounced write to DB

ON client_disconnect(socket, docId, userId):
    leaveRoom(socket, docId)
    broadcastToRoom(docId, 'user_left', userId)
    IF noActiveUsers(docId):
        DB.saveDocument(docId, Redis.get('doc:' + docId)) // final persist

```

4. Design Evaluation

a) Modularity

AdaptIQ is divided into six clearly scoped backend modules: Authentication, Quiz, Performance Analysis, Recommendation, Notes, and Collaboration. Each module owns its routes, service logic, and database models. On the frontend, React components are organized by feature (QuizView, ProgressDashboard, NotesExplorer, CollabEditor). This separation ensures that changes to one module do not cascade into others.

b) Maintainability and Extensibility

- New subjects and quizzes can be added directly via database records without code changes.
- New ML models can be swapped in by updating a single model-loader interface.
- New note file types can be supported by adding a handler in the Notes service.
- Frontend changes (e.g., redesigning the quiz UI) require no backend modifications and vice versa.

c) Scalability

- The stateless FastAPI backend can be horizontally scaled behind a load balancer.
- Redis handles session state and pub/sub, so multiple backend instances share consistent collaboration state.
- PostgreSQL supports read replicas for heavy read loads (notes search, progress queries).
- The ML engine can be extracted into a dedicated microservice if inference load grows.
- Docker containers make independent scaling of each layer straightforward.

d) Trade-offs

Trade-off	Description
Simplicity vs Flexibility	Monolithic layered architecture was chosen over microservices. This reduces DevOps overhead but means the full backend must be deployed for any change.
Redis memory vs persistence	Collaboration state is kept in Redis for speed, with periodic DB writes. A network failure before the final persist could lose a few seconds of edits.
ML accuracy vs latency	The recommendation model is pre-trained and served in-process for speed (< 5s response), but re-training requires a deployment cycle rather than live updates.
PostgreSQL vs Elasticsearch	Basic full-text search is handled by PostgreSQL to avoid infrastructure complexity. If the notes repository grows to millions of records, a dedicated search engine would be needed.

e) Alternative Designs Considered

Alternative	Why Considered	Why Rejected
Microservices Architecture	Better isolation, independent deployment of ML, notes, collab services.	Over-engineering for a student project; high DevOps complexity not justified at this scale.
MongoDB (NoSQL) as primary DB	Flexible schema for varied note metadata and quiz structures.	Relational integrity is important; PostgreSQL JSON columns handle flexible metadata where needed.
Firebase Realtime DB for collab	Managed real-time sync out-of-the-box, no WebSocket server needed.	Introduces a third-party dependency, limits control over data and pricing, conflicts with self-hosted design goals.